



YA2YOS · KERNEL ENGINEERING

Ya2yOS

内核设计与工程实践

Rust 宏内核实现总览

RISC-V64

LoongArch64

Linux ABI

参赛队员：饶晓杰 指导老师：杨磊

syscall · task · scheduler · mm · VFS · signal · net · arch



CONTENTS

目录

从系统定位出发，沿实现路径走向可复核成果

01 · 系统定位

Rust 宏内核、Linux ABI、RISC-V64 / LoongArch64；面向真实 Linux 用户态路径与工具链负载。

02 · 系统介绍

从系统架构图出发，依次介绍进程管理、多核调度、内存管理、信号机制、网络模块和设备驱动。

03 · 关键增量

多核运行、CFS 调度、StarryOS 网络移植、uaccess、文件锁、syscall 扩展与模块重构。

04 · 发展规划

扩展文件系统类型、提升文件 I/O、持续丰富网络模块，并完成开发板实机运行与验证。

关键词：兼容边界 · 状态机 · 并发控制 · 证据边界



CHAPTER 01

系统定位

Rust 宏内核 · Linux ABI · RISC-V64 / LoongArch64



项目定位：Ya2yOS 是什么？

面向真实 Linux 用户态负载的 Rust 宏内核

Rust

内核主体 · 类型安全 · 显式所有权



宏内核

task · mm · fs · net · signal 共享地址空间



Linux ABI

syscall · errno · fd · signal · mmap

我们要解决什么问题

让 glibc / musl、BusyBox、LTP 子集和 Rust 工具链等真实用户态程序，在自研内核上沿 Linux ABI 路径运行，而不是停留在“能启动、能调用”的接口演示。

系统边界

Ya2yOS 直接运行在 QEMU / 板级硬件上，不是用户态模拟器；以 Linux 用户可见语义为兼容边界，关注 syscall、errno、fd、阻塞 / 唤醒和资源释放。

目标平台与负载

支持 RISC-V64 / LoongArch64，面向高并发 fork / exec / mmap、真实 ext4、SMP 共享地址空间和双架构 QEMU 验证场景。

定位：用 Rust 构建可运行、可扩展、可验证的 Linux ABI 宏内核



设计亮点：把复杂性收敛到可解释的内核路径

不仅实现接口，更统一对象、状态和资源生命周期

01 · Rust 所有权贯穿生命周期

任务、地址空间、文件和 socket 以引用计数与显式所有权管理；fork / exec / exit / close 的共享、复制和回收边界清晰。

02 · 统一任务状态机

RUNNING → BLOCKED → WAKING → RUNNABLE；
futex、pipe、socket、epoll、文件锁和定时器共享
park / wake。

03 · uaccess 快路径

copy_from_user / copy_to_user 优先走对齐、单页和权限已知的快速复制；异常路径保留EFAULT 语义。

04 · MailBox 核间通信

TLB shutdown、任务唤醒和同步请求通过 MailBox 发布，由 IPI 通知目标 Hart 并收集 ACK。

05 · 架构差异硬件层收敛

RISC-V64 与 LoongArch64 的 trap、页表 / TLB、
timer、IPI 和 VirtIO 由架构层适配，上层共享内核语义。

06 · 正确性与性能并重

先保证 Linux 语义、锁序和可唤醒路径，再通过页缓存、read_at、COW、调度和可观测性优化。

设计主线：薄入口 → 快路径 → MailBox / IPI 协议 → 内核对象 → 状态转换 → 资源回收



CHAPTER 02

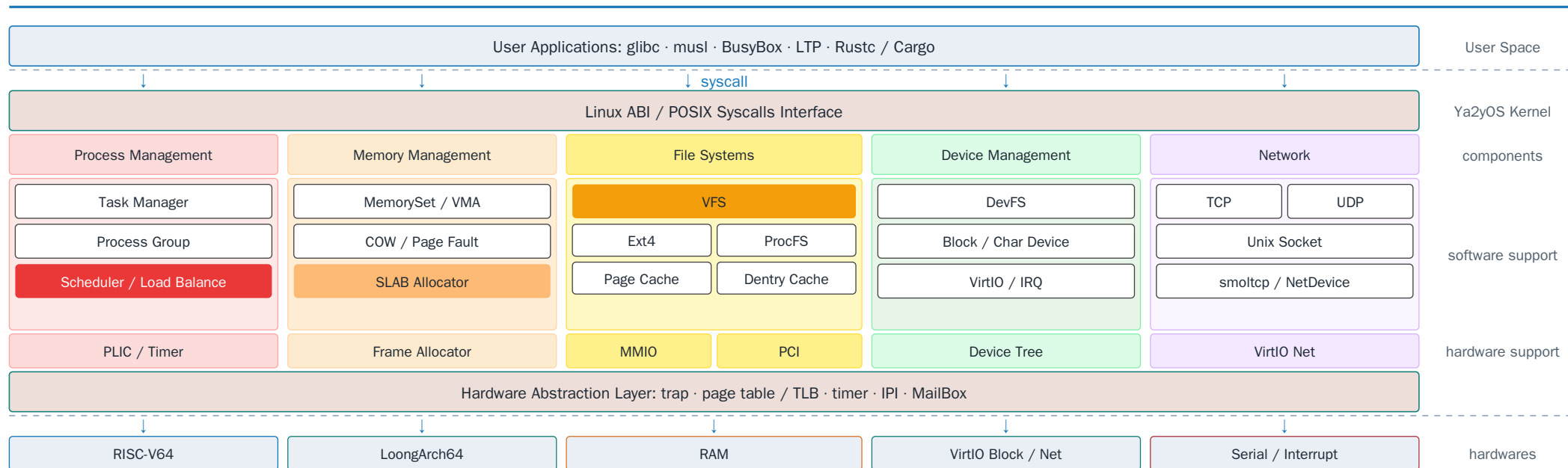
系统介绍

从系统架构图出发，沿进程、调度、内存、信号、网络与驱动走完整实现路径



系统架构

五类内核组件由 Linux ABI 统一入口衔接，并通过 HAL 收敛架构与设备差异





进程管理：对象、资源与生命周期

以 ProcessControlBlock 和 TaskControlBlock 建立进程语义与线程执行边界





多核调度：让可运行任务到达合适的 Hart

共享 CFS 就绪队列与线程 affinity 共同决定任务选择和远端唤醒



共享就绪队列

CFS 从同一 ready queue 选择任务；任务状态由原子字段或任务锁保护，避免队列选择与状态转换脱节。

亲和性与唤醒

TCB 保存 Linux 可见的 CPU affinity mask；调度器保留暂不满足 affinity 的任务，空闲目标 Hart 可由平台唤醒。

当前边界

已实现 online-Hart mask、affinity 过滤和 idle-Hart 唤醒；运行中迁移、per-CPU queue 与 work stealing 仍属于后续演进。

调度目标：可运行性正确优先，再通过拓扑与队列策略提升并行度



内存管理：从 VMA 到跨核页表一致性

MemorySet 管理地址空间；缺页、COW 与 TLB shutdown 共同维护映射语义



地址空间对象

MemorySet 以 RwLock 包装 MemorySetInner；VMA、页表与映射生命周期集中管理，文件映射保留 backing inode。

缺页与 COW

文件页先在锁外获取，再在写锁阶段安装；私有可写映射在 fork 后保留 COW 语义，写入时才复制物理页。

跨核一致性

PTE 替换遵循 UPDATE_LOCK → MemorySet 写锁顺序；remote TLB 经 MailBox / IPI 收集 ACK，收敛前不释放旧帧。

用户访问边界

copy_from_user / copy_to_user 与页表检查共同保证 uaccess 失败返回 EFAULT，而不是越界访问内核内存。



信号机制：从 pending 集到用户态处理函数

信号投递、选择、用户栈 frame 构造与 rt_sigreturn 构成完整闭环



投递与唤醒

信号写入目标线程或线程组的 pending 集；若任务处于可中断阻塞状态，投递路径将其置为可运行，以便及时消费。

选择语义

pending 模块在 trap 返回前选择未被 mask 的信号；标准信号重复到达时保留 pending 位与首次 siginfo_t，不无限排队。

用户态 frame

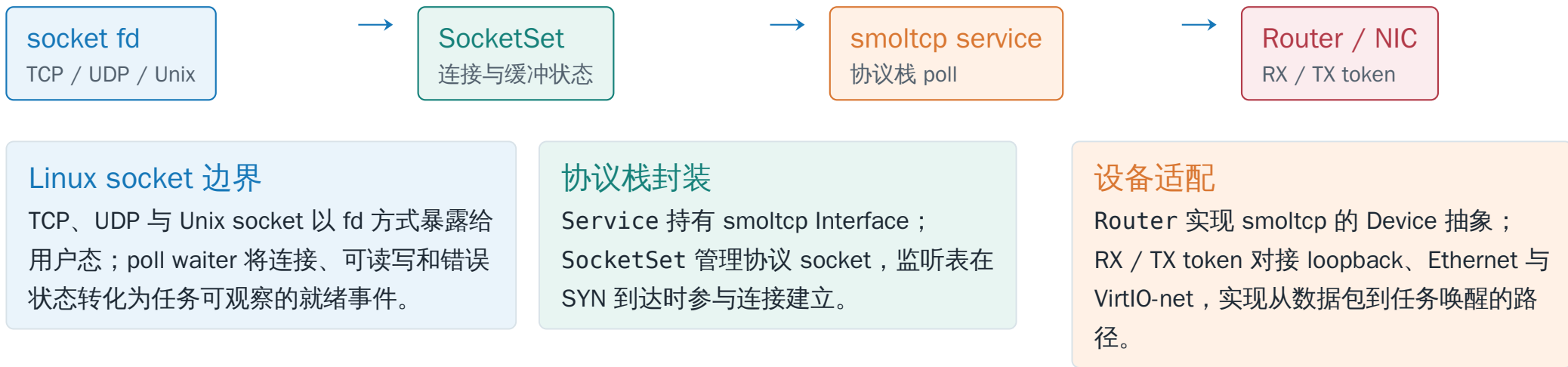
有 handler 时在用户栈构造 signal frame；rt_sigreturn 校验 frame magic 与布局后恢复寄存器、PC 和栈指针，非法 frame 返回 EINVAL。

信号既是异步通知，也是从任务状态、用户内存到 trap 返回路径的完整协议



网络模块：socket 语义落到 smoltcp 与网卡收发

系统调用创建文件描述符；协议状态机与底层设备通过统一 poll / RX / TX 路径衔接



socket readiness → poll waiter → packet RX / TX → wake task



设备驱动：把架构差异收敛为块设备与网卡接口

RISC-V64 与 LoongArch64 选择不同总线传输方式，上层文件系统和协议栈共享设备语义



双架构适配

RISC-V64 使用 VirtIO-MMIO 绑定；
LoongArch64 通过 PCI transport 发现和绑定 VirtIO 能力。上层仅依赖 BlockDeviceImpl / NetDeviceImpl。

块设备路径

块设备实现封装 VirtIO 队列与同步访问；文件系统经统一块接口提交读写，不依赖具体 MMIO 寄存器或 PCI 配置空间。

网卡路径

VirtIO-net 预置接收描述符并维护发送缓冲；队列访问串行化，随后将收发事件交给网络模块的 RX / TX 路径。

平台发现与传输层可变，块设备 / 网卡接口稳定，上层内核语义保持共享



CHAPTER 03

关键增量工作

并发运行 · Linux 语义 · 网络移植 · 工程结构



关键增量工作：围绕并发、兼容与工程化

七项工作共同把可启动的内核基础推进为能承载真实用户态路径的系统

01 · 多核运行

维护 online-Hart 状态、核间唤醒和共享地址空间的跨核可见性。

02 · CFS 调度

共享 ready queue、线程 affinity 过滤与 idle-Hart 唤醒形成闭环。

03 · StarryOS 网络

移植网络模块与 NetDriverOps 抽象，接入 smoltcp 与 VirtIO-net。

04 · uaccess

copy_from_user / copy_to_user 提供架构快路径、跨页安全路径和 EFAULT 语义。

05 · 文件锁

细化 POSIX/OFD/lease 锁、阻塞等待、死锁检测和 close/exit 回收。

06 · syscall

扩展 task、mm、fs、net、同步与 I/O multiplexing 的 Linux 接口。

07 · 模块重构

以职责拆分 signal、MemorySet、syscall、网络服务和驱动，提升内聚度。

增量主线：先让多核与资源正确运行，再扩展 ABI 覆盖并收敛模块边界



实现多核运行与 CFS 调度

从任务入队到目标 Hart 执行，在线拓扑、affinity、队列和唤醒路径必须保持一致



多核运行基础

维护 online-Hart mask；共享地址空间的 PTE 更新通过 remote TLB MailBox / IPI 收集 ACK，避免其他 Hart 使用失效映射。

CFS 可运行性

CFS 在共享 ready queue 中选择任务；TCB 持有 Linux 可见的 CPU affinity，暂不满足条件的任务保留在队列而不是错误执行。

唤醒与边界

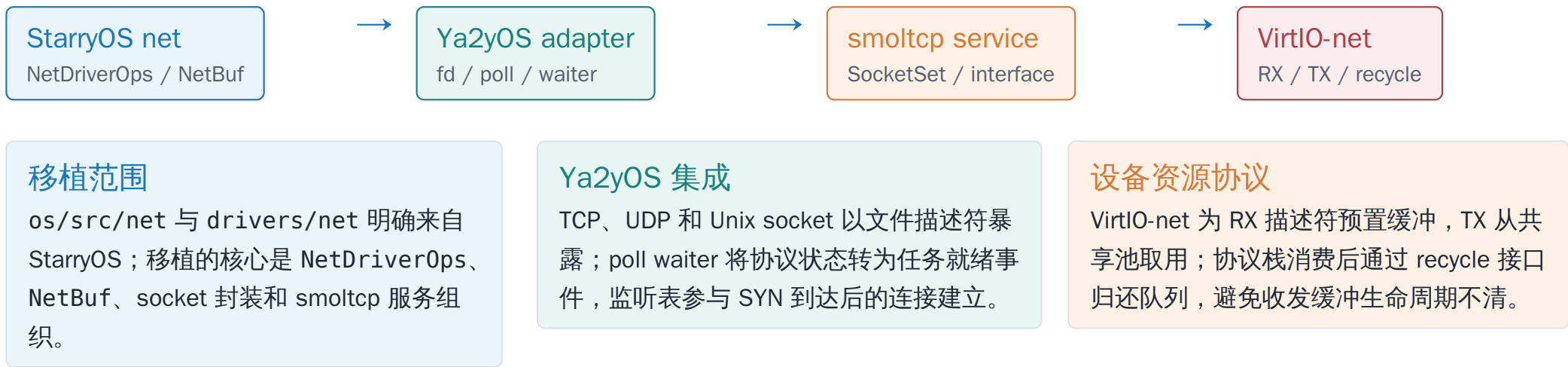
任务变为 ready 后按 affinity 通知可运行 Hart；空闲核可被平台唤醒。运行中迁移、per-CPU queue 与 work stealing 仍是后续工作。

SMP 的完成标准不是“启动多个核”，而是任务放置、TLB 一致性与唤醒路径均正确



移植 StarryOS 网络模块并接入 Ya2yOS

保留 StarryOS 的网络抽象优势，在 Ya2yOS 的任务、fd 和双架构驱动边界中重新落地



移植不是复制目录：必须把网络对象、任务唤醒和设备缓冲区所有权接入本内核语义



uaccess 双路径与文件系统锁细化

用户指针访问与文件并发控制都必须同时满足常见路径效率和异常路径的 Linux 语义



copy_from_user / copy_to_user 快路径

translate.rs 在长度、用户范围和当前地址空间满足条件时调用架构 copy_from_user / copy_to_user；短复制避免逐页通用翻译开销。

安全与异常路径

跨页、延迟分配、文件页和 COW 回到通用映射路径；uaccess scope 与 trap fixup 处理内核态访问异常，最终向 syscall 传播 EFAULT。

细化用户可见文件锁

file_lock 分离 POSIX record lock、BSD flock、OFD lock 与 lease；F_SETLKW 支持等待图死锁检测、waker 注册和 close/exit 自动释放。

保守的内部锁边界

lwext4 C API / bcache 的内部并发仍采用可等待的准入保护；先保证一致性与锁序，再逐步评估资源级细粒度并发。



丰富 syscall，并重构模块边界

扩大 ABI 覆盖面时，入口保持薄，核心语义与高风险状态机回到各自子系统



syscall 功能扩展

覆盖 clone3、rseq、key、IPC、xattr、inotify / fanotify、socket 消息收发、epoll / poll、timerfd 等更多真实用户态路径。

按领域拆分入口

syscall 划分 task、mm、fs、net、sync、io_mpx、ipc、time、sys 等目录；handler 负责解码、复制、查表和 errno，语义不堆积在大分发函数。

提高内聚度

signal 拆为 delivery/pending/frame/timer；MemorySet 拆分 handle、pagefault、fork_clone 等；网络服务、驱动和文件锁也按资源职责拆分。

工程目标：扩展接口不制造巨型模块，让每条 ABI 路径都落到可维护的对象、锁和生命周期



CHAPTER 04

发展规划

文件系统扩展 · I/O 优化 · 网络完善 · 开发板实机运行



发展规划：从可运行走向更完整的系统能力

围绕存储、网络与实机部署持续扩展，所有优化以语义正确和可复核验证为前提

01 · 支持更多文件系统

在统一 VFS、dentry/inode 和 mount 语义下接入更多文件系统类型；完善不同文件系统的路径解析、权限、元数据和挂载参数兼容性。

02 · 提高文件系统 I/O 速率

围绕 PageCache 命中、顺序/批量读写、跨页复制和块设备提交路径减少重复工作；使用同配置、成功收尾的基准和文件系统回归验证优化收益。

03 · 继续丰富网络模块

补齐 socket 选项、协议语义、路由与设备事件处理；持续加强 TCP / UDP / Unix socket 与 poll / epoll、VirtIO-net 之间的一致性。

04 · 在开发板上成功运行

完成实机启动、内存与中断初始化、块设备/网卡驱动和串口观测；在开发板上跑通用户态程序、文件 I/O、网络通信与压力回归。

路线：VFS 扩展 → I/O 优化 → 网络完善 → 实机验证，逐步形成可用且可验证的操作系统



Ya2y0S

内核设计与工程实践

谢谢！